

Planning:

Organisation:

Responsibilities:

We have 2 people on our team. Jasper, the Programmer and Isaac, the Visual Designer. These are the main roles, and the rest are completed collaboratively.

Submission guidelines:

We will be checking our game against the rules every so often, ensuring that it fully complies with them, while also just going out of our way to make sure that we use royalty free music and SFX and all the obvious, easy to comply with, things.

Basic Game Idea:

In our game you are a seagull that can squawk at humans, steal chips, complete quests and play minigames. The objective is to complete all the quests and get the lowest score on minigames making it replayable.

Workflow:

We will start by making models on Magicavoxel. We will then give the models basic code and then create more models to fit with the seagull's basic code to create quests. We'll do things in this order because we think we'll find it easier to code scripts into models than just placeholder sprites.

Timeline:

We are aiming to have everything complete 2 weeks before the due date, so we have time to fix any errors that occur and keep the game polished. Jasper will work on code, while Isaac works a step ahead and builds more models for the game as decoration. The game needs to be ready for testing around 4 weeks before the due date. We will then leave the last 3 weeks for working on feedback and polish before submitting it before Wednesday, September 9th.

Inspiration and points of originality:

Untitled Goose Game and Goat Simulator are massive inspiration that this game takes from. The fun, 'annoying people' aspect is what we want this game to be based around. The quest system is a bit like the one from Goat Simulator where you can find secret quests or do the ones already given to you. Our game is more focused on the life of a seagull stealing chips and the voxel art style so looks completely different to Goat Sim and Untitled Goose Game. We feel our game will definitely be worth making, as even if

it's similar to those other games, it's still different enough to be worth it and can be easily spun off into a different game easily.

Technical Requirements:

Development Environment:

The finished game will run in both Windows and Mac, because we will build it in Godot. This game engine is a free and open-source engine meaning it's constantly being updated and is also what we have used for other projects. Other game engines like Unity have the asset store which are bonuses, and Godot is still an emerging engine, with less tutorials and a 3d engine not quite as built out as Unity's but Godot is simple and you don't need to bother with paying to release your game later, Unity's cryptic Error messages or difficult coding language. You will require an input device and something to play it on. Keyboard and mouse is standard, as controller support is present but not fully supported or recommended. If Available, speakers or headphones are also recommended to listen to the beautiful soundtrack. The game is not super resource heavy, but I do recommend a PC with a dedicated graphics card. My PC runs it at a standard 60 fps and drops to 30 when in or looking at the voxel ocean. I Tested it on my friends MacBook air, and it was considerably slower (30 fps and like 5ish fps when in water)

Resourcing/Capability:

We will be needing Godot the game engine and Magicavoxel the model editor that we are using for our voxel art. We will also need GitHub desktop setup on both computers that we are using for collaboration. We will learn new skills in every step of the process. We will learn more about coding in Godot and about how to make models in Magicavoxel. We will also take on new challenges and do things we haven't done in the game engine before. We will learn new skills through YouTube tutorials, Official Documentation, Online forums, AI code snippets, and just messing around with the programs

Designing:

Game overview:

Game Title:

Our game is called MSG (short for Maniac Seagull Game). We chose this name because it's perfect! You are a maniac seagull who loves to steal chips, and MSG is also related to monosodium glutamate (like the stuff in food) which has a mention in our game. This name tells people exactly what the game is about. You are a seagull that flies around and breaks stuff, some might say you're a maniac seagull.

Game Description:

In our game you are a seagull that is trying to make life as miserable for humans as possible. You fly around and complete quests and find new little secrets as you go and of course, poo and steal chips from humans. The end goal is to complete all the quests. Our game is intended for teenagers, kids and young adults. In our game you can fly around and just be yourself. Careful not to knock over the building, nudge nudge wink wink. You get to blow stuff up, knock stuff down, poo on people, complete quests, steal chips, have a dance, and meet a new secret friend. Our game has a bunch of NPC's. The Mafia leader is the quest giver; he gives you jobs to do to make the humans angry or uncover areas. The Humans, Essential to the game, they live up the world and provide a target for the seagull. Sean, Only uncovered later in the game Sean the sea monster can also be found in the game and spawned as a pet to follow you through the world with 'P'

Environment:

The game takes place at the beach in our world we built. It takes place in daylight and there are no extra weather mechanics. The ocean plays a large role in our world. In the world you can ride boats and float bobbing up and down in the water. There is also a lighthouse in our world that when light up will shine bright.

Theme:

This year's theme is oceans

In our game you are a seagull that flies around an ocean. There are a bunch of mechanics and things related to oceans and beaches. e.g. the fish and chip shop, the voxel ocean, the lighthouse, and the sea monster.

Gameplay/Mechanics:

Objectives/Goals:

The main goal of our game is to complete all the quests given to you by the seagull mafia lord. To win the game the player should aim to complete all the quests, however the fun does not end there, as there is no true ending to the game and players are free to do whatever they find fun. There are no "levels" in our game, but players are constantly given new quests to complete, which the players should try to complete.



Laying out game mechanics in whiteboard before building the game

Perspective:

Our game is a 3D third person game, with a similar perspective to games like Fortnite and Roblox.

Controls:

Players in our game use WASD to move around and can press space to jump. If space is pressed in the air, the player begins to fly. Holding shift while flying makes the seagull flap and increases your speed. There are also many other actions, such as pressing 'E' to squawk and annoy humans, 'F' to poop, which is used in many quests, 'P' to toggle pet (once unlocked), backspace to view quests and esc to unlock the mouse.

Instructions/Tutorials:

There is a full tutorial on all the necessary controls in the game, which is presented at the beginning and upon completion, players should know how to play.

Visual and Audio Design:

Style:

The visual style is supposed to be comedic and exciting. It is inspired by the comedy genre and is supposed to look modern. The art is inspired by games like Bloons TD, with very bright colours.

The visual design is linked to the ocean by emulating the feelings of being on the beach, and getting your chips stolen by seagulls. The two main songs in the game are very similar to our visual style, as the main song is very upbeat and fun, and is also catchy. The second song is played in the dance party room and has a very strong beat. It's even synced to the RGB lights in the room, which fits the visual style of a flashy rainbow disco. We did not create any of the audio in our game, but we made all the art and models. We did not make our own music because neither of us had much experience in

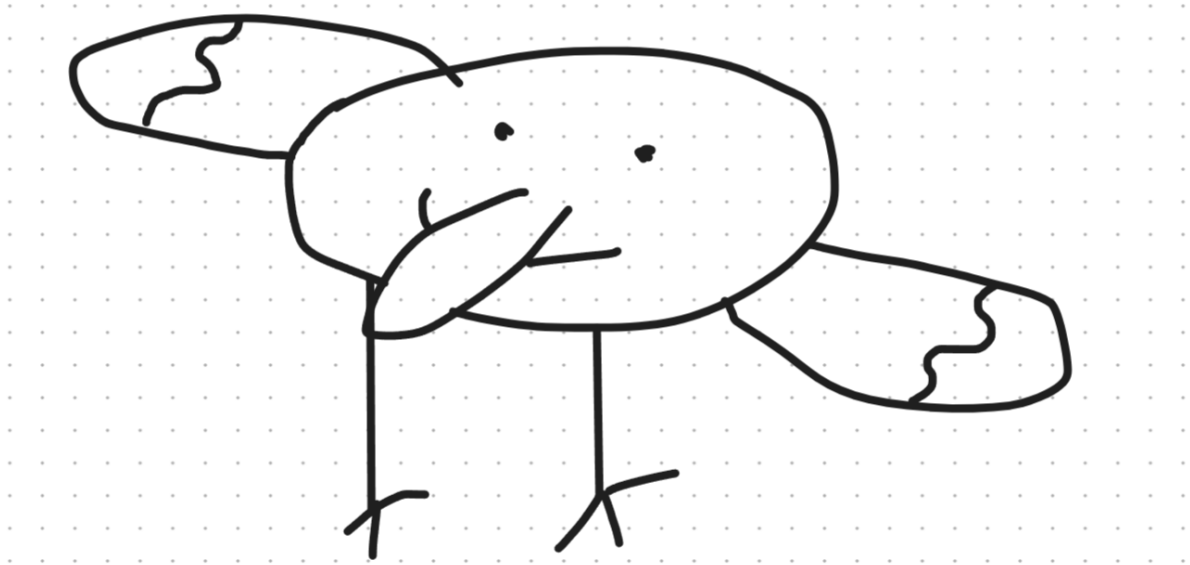
music creation, and there is much more good quality music that fits our visual theme on the internet than there are models that fit our unique visual style.



Screenshot of the whacky seagull world style from finished game



Screenshot of dance floor in finished game



Concept drawings created while planning

Process:

To achieve our visual style, we used bright colours, and made all our models on Magicavoxel, to get the pixels. It is also a lot easier than Blender. We did not create the audio on our game, and instead we went on royalty free download websites such as Pixabay and NCS. We specifically chose the songs: [whatdoyousee](#) by prodBigMike and Glitch Cat, available on NCS. [Too Fancy to Be Serious](#) by STAROSTIN available on Pixabay under the name: [Comedy Cartoon Funny Background Music](#). [LA LUNA \(Slowed\)](#) by VRZone available on Pixabay under the name: [Phonk](#). Whenever we had an idea for a model in the game, Isaac would usually look on Google for an image to base the model off. Usually making organic shapes (like a seagull) was hard, so we started with a basic rectangle or sphere, and added and removed voxels to make a more defined shape. Then we would paint it appropriate colours for the model.

Reflecting:

Testing:

Our game was tested by four people in our coding class. They were instructed to play the game and follow the tutorial and then give us suggestions on what to improve.

Fixing:

Many problems were found in testing, most complaints being about the boats and flying mechanics. We ended up making 'holding shift' not necessary to fly, only to fly faster, and we ended up fixing the boats, as pretty much everyone got softlocked when trying to use them.

Project execution:

We did finish our game and pretty much everything we wanted is in the game. There are some things we wanted that are not included in the final game, such as guns, skins, and minigames, however, everything we have really committed to is in the game.

Isaac Reflection:

Next time, I would like to improve my coding skill, as I did not do much coding in the game and mostly did art. This is because Jasper, the master coder, coded so many things that I was not really needed to do coding.

Me and jasper teaming up worked very well, as we basically had double the efficiency as if we worked separately, and we were able to meet up on weekdays to work on our game.

Next time, I would like to do more coding, and also try a 2D game, as it seems a lot simpler, as even some simple things like boats were hard to implement in 3D.

Jasper Reflection:

I would like to look more closely into the game engine for next time, as implementations worked, but were often built to just work and making different variants of things needed to be hard coded and were not easy to create.

I think our teamwork worked well, (and so did the dance floor that thing looks so cool!) We worked together a lot during this project, and it was nice having a bit of the weight off your back and someone you can have fun with while creating the game.

I would probably like to look more into Resources in Godot and maybe look at something that we haven't looked at yet. I would also like to try a different type of game, as I would quite like to make a puzzle game.

References

Inspiration Games:

- Untitled Goose Game (2019), House House. <https://goose.game>
- Goat Simulator (2014), Coffee Stain Studios. <https://store.steampowered.com/app/265930/>
- Bloons TD 6 (2018), Ninja Kiwi. <https://ninjakiwi.com/games/bloons-tower-defense/>

Software:

- Godot Engine 4.x - <https://godotengine.org> (MIT licence)
- MagicaVoxel, ephtracy - <https://ephtracy.github.io>
- GitHub Desktop - <https://desktop.github.com>

Audio (all royalty-free)

- whatdoyousee, prodBigMike and Glitch Cat - <https://ncs.io/whatdoyousee>.
- Comedy Cartoon Funny Background Music, STAROSTIN - <https://pixabay.com/music/comedy-comedy-cartoon-funny-background-music-492540/>.
- Phonk, VRZone - <https://pixabay.com/music/phonk-phonk-567414/>
- Sound effects: Pixabay — <https://pixabay.com/sound-effects/> (Pixabay Content Licence)

AI:

We used ai in our project for help creating scripts. And fixing GitHub sync issues that corrupted files. AI helped us with the ocean. Specifically trying to optimise it with LODs and fixing layering bugs. It also helped with floating, though I made the first floating code, AI fixed up some bugs with it. AI was used for the Megapoo cutscene. It helped with creating the code to build the layers of the cutscene, all the quest integration, and the poo physics was done by us, you can see package.gd for what the code looked like before the AI cutscene code was added. AI also helped, or more created, most of the quest code. This was before we had read the rules of the challenge, it has been edited by us, and all the quests have been added by us. It's just the quest manager. We did however make the quest complete screen and code ourselves. The Human script was made by us, with minimal debugging help from AI. The Player flying, walking and play animation mechanics were originally made by AI, but we fully rewrite the code after it got super cluttered, using the AI's flying maths and ideas and writing them ourselves again. All the other player squawking, pooing and cutscenes are all made by us. The debug console was made entirely with AI as it is not in the final game, and we needed it purely for testing purposes. I originally made all of the sound manager mechanics but just got ai to look over it and polish it up. (I made all the actual logic, and it added the push error.) If a script is not mentioned here, it means that we made it entirely or with very, very little help that it is not worth mentioning.

